

Regression

OLS, Diagnostics, Regularisation & Quant Finance Applications

The Workhorse Tool for Quantifying Relationships and Estimating Exposures

A Level 2 module in the Quantitative Finance Curriculum — revisited at Level 3

Covers: Simple/multiple linear regression · OLS · residuals / R^2 / adjusted R^2 · inference · multicollinearity · heteroskedasticity · autocorrelation · robust standard errors · Ridge / Lasso / Elastic Net · factor regression · alpha/beta estimation · pairs-trading applications

Table of Contents

How to Use This Guide

What You Need to Know Before Starting

Why It Matters

How This Connects to the Rest of the Curriculum

Glossary of Recurring Terms

Part 1 — Simple and Multiple Linear Regression

1.1 Simple Linear Regression and OLS

1.2 Multiple Linear Regression

1.3 Residuals, R^2 , and Adjusted R^2

1.4 Statistical Inference: Standard Errors, t-stats, and Confidence Intervals

Part 2 — Regression Diagnostics: When OLS Assumptions Break

2.1 Multicollinearity and the Variance Inflation Factor

2.2 Heteroskedasticity

2.3 Autocorrelation

2.4 Robust Standard Errors: White and HAC / Newey-West

Part 3 — Regularised Regression: The Bridge to Machine Learning

3.1 Ridge Regression (L2)

3.2 Lasso Regression (L1)

3.3 Elastic Net

Part 4 — Applications in Quantitative Finance

4.1 Factor Regression: Alpha and Beta Estimation

4.2 Pairs Trading and Hedge-Ratio Estimation

Practical Exercise

Common Beginner Mistakes

Advanced Extensions

Professional Applications

Where to Go Next

How to Use This Guide

This document is a companion to Regression, the second-level module in the quantitative finance curriculum that is revisited in more depth at Level 3. It covers the mechanics of ordinary least squares, the diagnostic checks that tell you when those mechanics are breaking down, the regularised variants that bridge into machine learning, and the two applications — factor regression and pairs trading — that make regression the single most-used statistical tool on a quant research desk.

The module is self-contained: every concept is introduced from first principles, paired with a short runnable Python sketch, and illustrated with a chart wherever a picture makes the idea land faster than an equation. Read it start to finish if regression is new to you, or jump straight to the section you need.

What You Need to Know Before Starting

- **Statistics (Level 1)** — descriptive statistics, probability distributions, and hypothesis testing (p-values, confidence intervals). Regression's inference machinery is built directly on these ideas.
- **Linear Algebra** — vectors, matrices, and matrix inversion. Multiple regression is most naturally expressed and solved in matrix form, and this module uses that notation throughout Section 1.2 onward.

***Tip** — If matrix notation is new, the intuition needed here is small: stacking every observation's regressors into rows of a matrix X and every outcome into a vector y turns 'solve n separate equations' into a single matrix equation, $\hat{\beta} = (X^T X)^{-1} X^T y$ — one formula that works no matter how many regressors there are.*

Why It Matters

Regression is the workhorse statistical tool for quantifying relationships between variables and estimating exposures. Nearly every other technique covered elsewhere in this curriculum either is a regression in disguise (CAPM and APT are both estimated by regression) or depends on one as an input (a pairs-trading hedge ratio, a factor loading, a risk model beta). It is also the natural on-ramp to machine learning: regularised regression — Ridge, Lasso, and Elastic Net — is usually the first model a quant encounters that trades a small amount of bias for a large reduction in variance, which is the central idea behind almost every more sophisticated learning algorithm that follows.

How This Connects to the Rest of the Curriculum

- **Research** — regression is the most common tool for signal validation, factor attribution, and hedge-ratio estimation. Before a new signal is taken seriously, a researcher will almost always regress it against known factors to check whether it is actually novel.

- **Development** — regression diagnostics (residual checks, VIF for multicollinearity) should be automated in any research pipeline, so that a broken assumption is flagged automatically rather than discovered after a strategy has already been sized and deployed.
- **Trading** — pairs-trading hedge ratios are typically estimated via rolling regression, re-estimated on a moving window so the hedge keeps up with a relationship that drifts over time (Section 4.2).

Glossary of Recurring Terms

Ordinary Least Squares (OLS) — The estimation method that chooses regression coefficients to minimise the sum of squared residuals — the vertical distances between observed and fitted values.

Residual — The difference between an observed value and the value the fitted regression predicts for it, $\varepsilon_i = y_i - \hat{y}_i$; the raw material for every diagnostic check in Part 2.

R² (coefficient of determination) — The fraction of total variance in the dependent variable explained by the regression, ranging from 0 (no explanatory power) to 1 (a perfect fit).

Adjusted R² — A version of R² that penalises adding regressors that don't genuinely improve the fit, making it safer than raw R² for comparing models with different numbers of predictors.

Multicollinearity — A condition where two or more regressors are highly correlated with each other, making it statistically difficult to isolate each one's individual effect on the dependent variable.

Variance Inflation Factor (VIF) — A diagnostic statistic, one per regressor, that quantifies how much multicollinearity has inflated that regressor's coefficient variance relative to a world with no correlation among predictors.

Heteroskedasticity — A condition where the variance of the regression's error term is not constant across observations, violating a standard OLS assumption and invalidating the usual standard errors.

Autocorrelation — Correlation between a regression's residuals and their own past values, common in time-series regressions and, like heteroskedasticity, a threat to the validity of standard errors rather than to the coefficient estimates themselves.

Robust / HAC standard errors — Standard errors computed with a formula that remains valid under heteroskedasticity (White / robust) or under both heteroskedasticity and autocorrelation (HAC, e.g. Newey-West), used in place of the default OLS standard errors when either problem is present.

Ridge regression (L2) — A regularised regression that shrinks coefficients toward zero by penalising the sum of their squared values, reducing variance at the cost of a small, deliberate bias.

Lasso regression (L1) — A regularised regression that penalises the sum of the absolute values of the coefficients, which — unlike Ridge — can shrink coefficients to exactly zero, performing automatic variable selection.

Elastic Net — A regularised regression combining both the L1 (Lasso) and L2 (Ridge) penalties, useful when predictors are both numerous and correlated.

Alpha (α) — In a factor regression, the intercept — the part of an asset's average return left unexplained by its factor exposures.

Beta (β) — In a factor regression, the slope — an asset's sensitivity to a given factor's return, most commonly the overall market.

Hedge ratio — The regression slope obtained from regressing one asset's returns on another's, used to size an offsetting position — most commonly in pairs trading (Section 4.2).

Part 1 — Simple and Multiple Linear Regression

This part covers the mechanics of ordinary least squares: how a line (or hyperplane) is fitted to data, how to judge whether that fit is any good, and how to attach a level of statistical confidence to the resulting coefficients. Everything in Parts 2 through 4 assumes this mechanical foundation.

1.1 Simple Linear Regression and OLS

Simple linear regression models a dependent variable y as a linear function of a single independent variable x , plus an error term that captures everything the line does not explain:

$$y = \alpha + \beta x + \varepsilon$$

Here α is the intercept (the predicted value of y when $x = 0$), β is the slope (how much y changes for a one-unit change in x), and ε is the error term — assumed, under the classical OLS assumptions, to have mean zero and to be uncorrelated with x . Ordinary Least Squares chooses the estimates $\hat{\alpha}$ and $\hat{\beta}$ that minimise the sum of squared residuals across all n observations:

$$\text{minimise } \sum_i (y_i - \alpha - \beta x_i)^2 \Rightarrow \hat{\beta} = \text{Cov}(x, y) / \text{Var}(x), \quad \hat{\alpha} = \bar{y} - \hat{\beta} \bar{x}$$

Squaring the residuals rather than, say, taking absolute values is not an arbitrary choice: it makes the minimisation problem solvable in closed form (no iterative search required), and it penalises large errors disproportionately, which is usually — though not always — the desired behaviour.

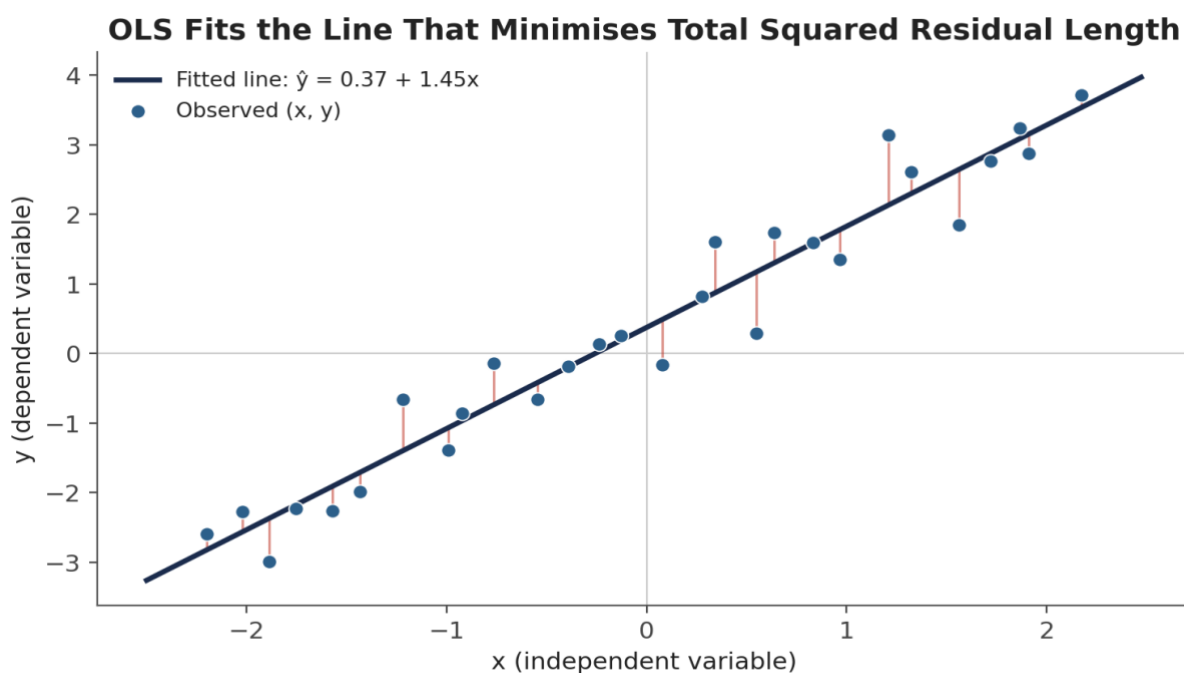


Figure 1.1 — OLS fits the line that minimises the sum of squared residuals (red segments), not the sum of their absolute values or any other loss function.

```
import numpy as np
```

```
# beta, alpha via simple linear regression (least squares)
beta, alpha = np.polyfit(x, y, deg=1)
y_hat = alpha + beta * x
residuals = y - y_hat
```

Assumptions to keep in mind — OLS is only guaranteed to be the 'best' (minimum-variance, unbiased) linear estimator — the Gauss-Markov theorem — when the relationship is genuinely linear, the errors have mean zero and constant variance, the errors are uncorrelated with each other and with x , and (for exact small-sample inference) the errors are normally distributed. Part 2 is built entirely around what happens, and what to do, when these assumptions fail.

1.2 Multiple Linear Regression

Multiple regression extends the same idea to several independent variables at once, which is the form almost every real application actually uses — a stock's return depends on more than one factor, and a hedge ratio may need to control for more than one instrument:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_k x_k + \varepsilon$$

Stacking every observation into a matrix makes the problem compact: X is an $n \times (k+1)$ matrix (one row per observation, one column per regressor, plus a column of ones for the intercept), y is an $n \times 1$ vector of outcomes, and β is the $(k+1) \times 1$ vector of coefficients to be estimated. The OLS solution generalises cleanly:

$$y = X\beta + \varepsilon \Rightarrow \hat{\beta} = (X^T X)^{-1} X^T y$$

Each coefficient β_j is now interpreted as a partial effect: the expected change in y for a one-unit change in x_j , holding every other regressor in the model fixed. This 'holding constant' interpretation is what makes multiple regression so useful for isolating one factor's contribution from another's — and it is also exactly what breaks down under severe multicollinearity (Section 2.1), since it is hard to hold one regressor fixed while varying another that moves in lockstep with it.

```
import statsmodels.api as sm

X = factor_data[['Mkt-RF', 'SMB', 'HML']]
X = sm.add_constant(X) # adds the intercept (alpha) term
model = sm.OLS(y, X).fit()
print(model.params)    # alpha and the three factor betas
```

1.3 Residuals, R^2 , and Adjusted R^2

Every fitted regression decomposes the total variation in y into a piece explained by the model and a piece left over in the residuals. Writing TSS for total sum of squares, ESS for explained sum of squares, and RSS for residual sum of squares:

$$TSS = \sum (y_i - \bar{y})^2 = ESS + RSS = \sum (\hat{y}_i - \bar{y})^2 + \sum (y_i - \hat{y}_i)^2$$

R^2 is simply the fraction of total variance the model explains:

$$R^2 = ESS / TSS = 1 - (RSS / TSS)$$

R^2 has one mechanical property that makes it dangerous to use carelessly: adding any additional regressor, however irrelevant, can never decrease it, and will almost always increase it slightly by chance alone. Adjusted R^2 corrects for this by penalising extra regressors relative to the sample size n and the number of regressors k :

$$\bar{R}^2 = 1 - (1 - R^2) \times (n - 1) / (n - k - 1)$$

Adjusted R^2 only rewards a new regressor if it improves the fit by more than would be expected by chance, which makes it the safer statistic for comparing two models with different numbers of predictors.

Common mistake — Trusting R^2 alone without checking residual diagnostics. A regression can have a high R^2 and still badly violate the assumptions behind valid inference — heteroskedastic or autocorrelated residuals do not necessarily lower R^2 , but they do invalidate the standard errors built on top of it (Sections 2.2 and 2.3). R^2 measures fit; it says nothing about whether the fit is estimated reliably.

1.4 Statistical Inference: Standard Errors, t-stats, and Confidence Intervals

A point estimate $\hat{\beta}$ is only half the picture — it comes from a finite sample and would come out slightly differently with a different sample. The standard error $se(\hat{\beta})$ quantifies that sampling variability, and it is the basis for every inferential statement about the coefficient:

$$t = \hat{\beta} / se(\hat{\beta}) \quad 95\% \text{ CI: } \hat{\beta} \pm 1.96 \times se(\hat{\beta}) \quad (\text{large-sample approximation})$$

The t-statistic tests the null hypothesis that the true coefficient is zero — that is, that x has no linear relationship with y once the other regressors are accounted for. A t-statistic with magnitude above roughly 2 (for reasonably large samples) corresponds to a p-value below the conventional 5% threshold, meaning the estimated coefficient is unlikely to have arisen by chance if the true coefficient were actually zero.

Worked example. A regression of a stock's excess return on the market's excess return produces $\hat{\beta} = 1.35$ with a standard error of 0.28. The t-statistic is $1.35 / 0.28 \approx 4.8$ — comfortably beyond the ± 2 rule of thumb, so the market exposure is statistically significant at any conventional threshold. The 95% confidence interval, $1.35 \pm 1.96 \times 0.28$, runs from roughly 0.80 to 1.90: the data are consistent with a true beta anywhere in that range, not just the single point estimate of 1.35.

Tip — Statistical significance is not the same as economic significance. A beta of 0.02 can be highly statistically significant in a large enough sample while being far too small to matter for portfolio construction or risk management. Always look at the magnitude of a coefficient alongside its significance, not instead of it.

Part 2 — Regression Diagnostics: When OLS Assumptions Break

OLS is the Best Linear Unbiased Estimator (BLUE) only when the Gauss-Markov assumptions hold: a genuinely linear relationship, no perfect multicollinearity among regressors, errors with constant variance (homoskedasticity), and errors uncorrelated with each other (no autocorrelation). This part covers how each assumption can fail, how to detect the failure, and — critically — what actually goes wrong when it does. It is worth stating up front what does not go wrong: violating homoskedasticity or the no-autocorrelation assumption does not bias the coefficient estimates themselves. It biases the standard errors, and therefore every t-stat, p-value, and confidence interval built on top of them.

2.1 Multicollinearity and the Variance Inflation Factor

Multicollinearity occurs when two or more regressors are highly correlated with each other. In the extreme case of perfect correlation, $(X^T X)$ becomes non-invertible and OLS has no unique solution at all; in the much more common case of high-but-imperfect correlation, a solution exists but the individual coefficients become unstable — small changes in the data can swing a coefficient's estimated value, and even its sign. The standard diagnostic is the Variance Inflation Factor, computed per regressor by regressing that regressor on all the others and taking its own R^2 from that auxiliary regression:

$$VIF_i = 1 / (1 - R_i^2)$$

A VIF of 1 means no inflation at all (the regressor is uncorrelated with the others); a commonly used rule of thumb treats VIF above 5 as worth investigating and VIF above 10 as a serious problem.

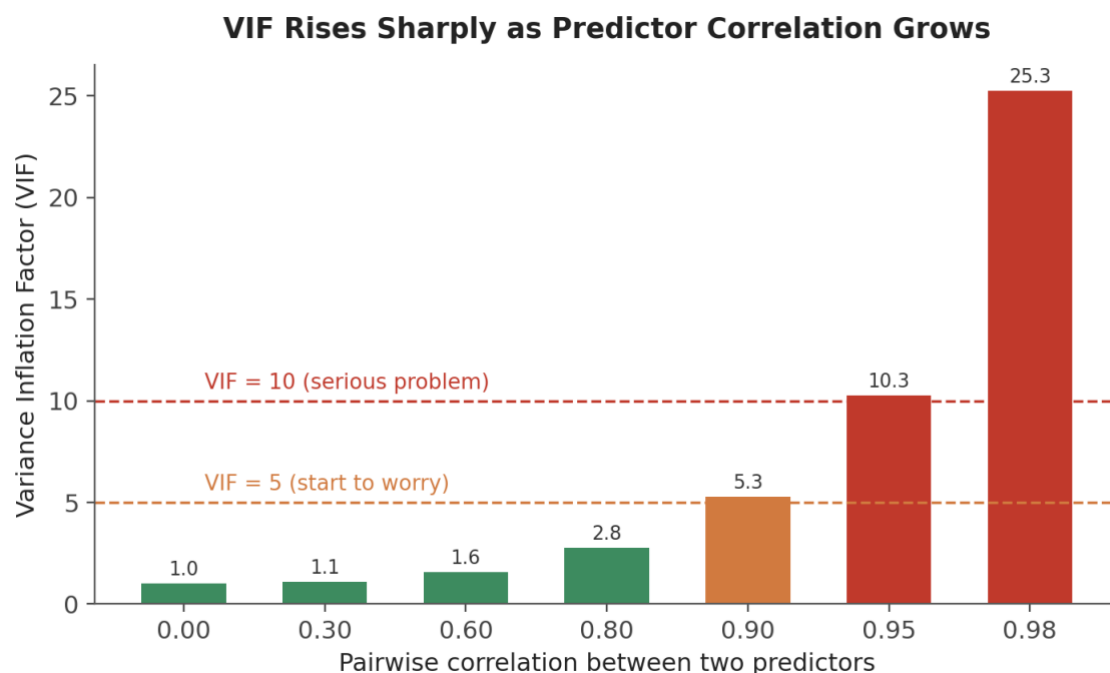


Figure 2.1 — VIF rises sharply, not linearly, as pairwise correlation between two predictors approaches 1. Below about 0.8 correlation, inflation is modest; above about 0.9, it accelerates quickly.

```
from statsmodels.stats.outliers_influence import variance_inflation_factor

vif = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
pd.Series(vif, index=X.columns)
```

What multicollinearity does not break — A model with severe multicollinearity can still predict y accurately; the fitted values and overall R^2 are unaffected. What breaks down is the ability to attribute the outcome to any one correlated regressor individually — exactly the attribution problem that factor regression (Section 4.1) exists to solve, which is why factor models are careful to choose factors that are not too highly correlated with each other.

2.2 Heteroskedasticity

Heteroskedasticity means the variance of the error term is not constant across observations — commonly, errors grow larger as the fitted value grows larger, which is typical of financial return data where larger positions or more volatile regimes produce larger absolute pricing errors. The standard visual diagnostic plots residuals against fitted values and looks for a fan or cone shape:

Residuals vs. Fitted Values: The Standard Heteroskedasticity Check

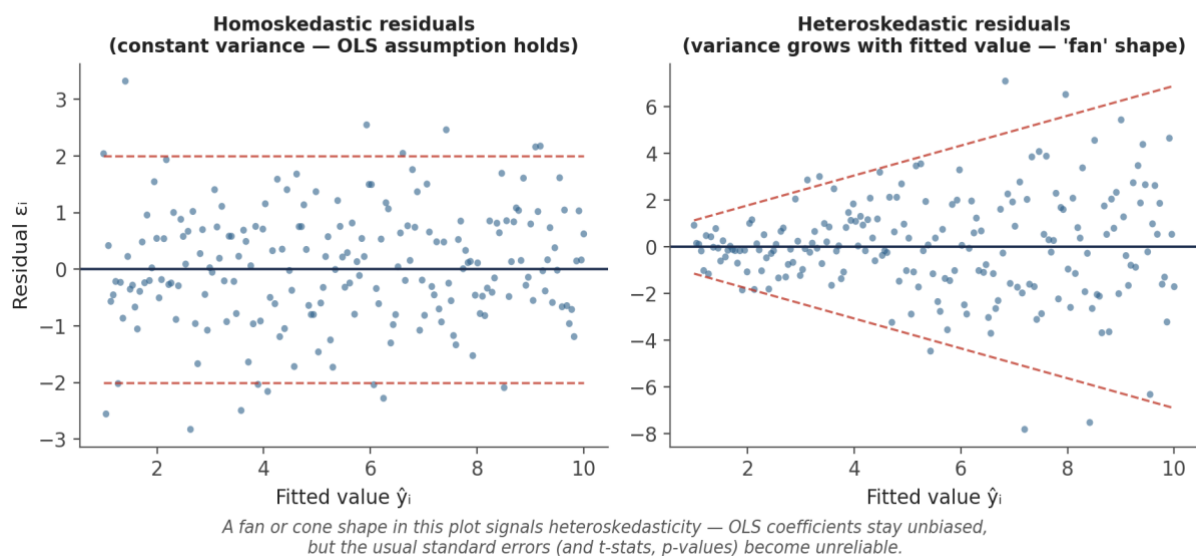


Figure 2.2 — Residuals vs. fitted values. Left: constant spread (homoskedastic, the OLS assumption holds). Right: spread widening with the fitted value (heteroskedastic) — a classic fan shape.

Formal tests exist alongside the visual check — the Breusch-Pagan and White tests both regress the squared residuals on the original regressors (or their cross-products, for White) and test whether that auxiliary regression has any explanatory power.

```
import statsmodels.stats.api as sms

bp_test = sms.het_breuschpagan(model.resid, model.model.exog)
labels = ['LM stat', 'LM p-value', 'F stat', 'F p-value']
dict(zip(labels, bp_test))
```

2.3 Autocorrelation

Autocorrelation (or serial correlation) occurs when a regression's residuals are correlated with their own past values — the hallmark case being a time-series regression where an unmodelled trend, seasonality, or momentum effect leaks into the errors instead of the fitted line. The standard visual diagnostic is the autocorrelation function (ACF) of the residuals:

Autocorrelated Residuals: A Time-Series Regression Diagnostic

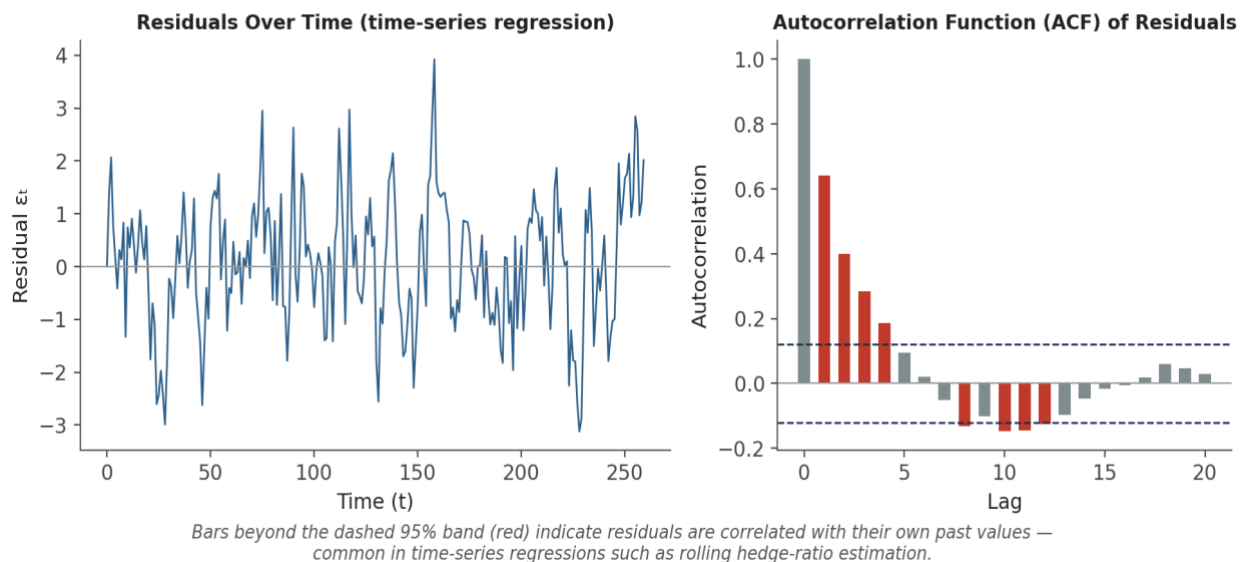


Figure 2.3 — Left: residuals from a time-series regression plotted over time, showing visible persistence. Right: the ACF of those residuals — bars beyond the dashed 95% band (shown in red) indicate statistically significant autocorrelation at that lag.

The Durbin-Watson statistic is the traditional formal test for first-order autocorrelation, ranging from 0 to 4 with a value near 2 indicating no autocorrelation, values below 2 indicating positive autocorrelation, and values above 2 indicating negative autocorrelation:

```
from statsmodels.stats.stattools import durbin_watson

dw = durbin_watson(model.resid)  # ~2.0 = no autocorrelation
```

Common mistake — Ignoring autocorrelated residuals when computing standard errors on time-series data. Rolling hedge-ratio regressions, factor regressions on overlapping return windows, and almost any regression on daily or monthly financial time series are prone to autocorrelated residuals. Using default OLS standard errors in this setting typically understates the true standard errors, making coefficients look more statistically significant than they actually are — the fix is to use robust/HAC standard errors, covered next.

2.4 Robust Standard Errors: White and HAC / Newey-West

The practical fix for both heteroskedasticity and autocorrelation is not to change the coefficient estimates — OLS coefficients remain unbiased under both problems — but to change how their

standard errors are computed. White (heteroskedasticity-robust) standard errors adjust for non-constant error variance; Heteroskedasticity and Autocorrelation Consistent (HAC) standard errors, most commonly the Newey-West estimator, adjust for both problems simultaneously and are the standard choice for time-series regressions in finance:

```
import statsmodels.api as sm

# Heteroskedasticity-robust (White) standard errors
model_robust = sm.OLS(y, X).fit(cov_type='HC1')

# HAC / Newey-West standard errors (heteroskedasticity + autocorrelation)
model_hac = sm.OLS(y, X).fit(cov_type='HAC', cov_kws={'maxlags': 4})
```

Tip — Newey-West requires choosing a maximum lag length (how many periods of autocorrelation to correct for). A common rule of thumb is roughly $4 \times (T/100)^{2/9}$ for T observations, but in practice it is worth checking that results are not overly sensitive to the specific lag choice — try a couple of reasonable values and confirm the conclusion doesn't change.

Part 3 — Regularised Regression: The Bridge to Machine Learning

Ordinary least squares is unbiased, but when regressors are numerous, correlated, or the sample is small relative to the number of predictors, its variance can be large enough that the resulting coefficients are unreliable even though they are, on average, correct. Regularisation trades a small, deliberate amount of bias for a often much larger reduction in variance — the same bias-variance trade-off that underlies essentially every machine learning model covered later in the curriculum, which is why this section is usually the first genuine bridge from classical statistics into ML.

3.1 Ridge Regression (L2)

Ridge regression adds a penalty proportional to the sum of squared coefficients to the OLS objective function:

$$\text{minimise } \sum_i (y_i - X_i\beta)^2 + \lambda \sum_i \beta_i^2$$

The regularisation strength λ controls the trade-off: $\lambda = 0$ recovers ordinary OLS exactly, and as λ grows every coefficient is shrunk progressively toward — but never all the way to — zero. Ridge is particularly effective when regressors are correlated, since it distributes the coefficient 'load' across correlated predictors rather than letting OLS arbitrarily favour one over another.

3.2 Lasso Regression (L1)

Lasso replaces Ridge's squared penalty with a penalty on the sum of absolute coefficient values:

$$\text{minimise } \sum_i (y_i - X_i\beta)^2 + \lambda \sum_i |\beta_i|$$

The geometry of the absolute-value penalty (as opposed to the squared penalty) means Lasso can, and typically does, shrink some coefficients to exactly zero rather than merely toward it. This gives Lasso a second job beyond shrinkage: automatic variable selection, dropping regressors it judges uninformative entirely out of the model.

Ridge vs. Lasso: How Coefficients Respond to Regularisation Strength

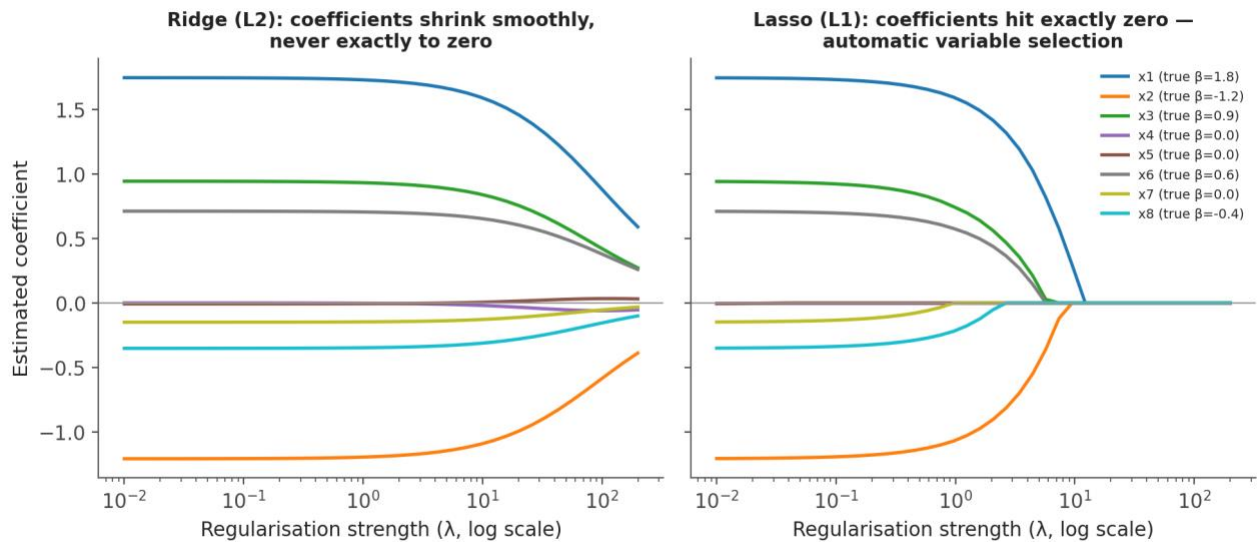


Figure 3.1 — Coefficient paths for the same simulated data (eight candidate predictors, three with a true coefficient of exactly zero) as regularisation strength λ increases. Ridge (left) shrinks every coefficient smoothly; Lasso (right) drives the zero-truth coefficients to exactly zero.

```
from sklearn.linear_model import Ridge, Lasso, ElasticNet
from sklearn.preprocessing import StandardScaler
```

```
Xs = StandardScaler().fit_transform(X) # regularisation requires scaled inputs
ridge = Ridge(alpha=1.0).fit(Xs, y)
lasso = Lasso(alpha=0.1).fit(Xs, y)
```

Why scaling matters — Ridge and Lasso penalise the raw magnitude of coefficients, so a regressor measured in units that make its natural coefficient large (or small) will be penalised more (or less) than an equally important regressor on a different scale. Standardising every predictor to mean zero and unit variance before fitting is standard practice, not an optional step.

3.3 Elastic Net

Elastic Net combines both penalties, controlled by a mixing parameter (commonly called α or l1_ratio , distinct from the regression intercept α used elsewhere in this document) that interpolates between pure Ridge and pure Lasso:

$$\text{minimise } \sum_i (y_i - X_i\beta)^2 + \lambda [\rho \sum_i |\beta_i| + (1 - \rho) \sum_i \beta_i^2]$$

Elastic Net is particularly useful when predictors are both numerous and correlated — a setting where pure Lasso tends to arbitrarily pick one of a group of correlated predictors and zero out the rest, while Elastic Net's Ridge component encourages correlated predictors to be kept, or dropped, together.

Method	Penalty	Coefficients hit zero?	Best suited to
OLS	None	No	Few, roughly uncorrelated regressors
Ridge (L2)	$\sum \beta_j^2$	No — shrinks smoothly	Correlated regressors; keep all predictors
Lasso (L1)	$\sum \beta_j $	Yes — automatic selection	Many candidate predictors, expect sparsity
Elastic Net	Mix of L1 and L2	Yes, in correlated groups	Many, correlated candidate predictors

Table 3.1 — Choosing among OLS, Ridge, Lasso, and Elastic Net.

Part 4 — Applications in Quantitative Finance

The two applications below are, in practice, the reason most quants learn regression in the first place: explaining where a return came from (factor regression), and sizing an offsetting position against a correlated instrument (pairs trading). Both are direct, practical uses of everything covered in Parts 1 and 2.

4.1 Factor Regression: Alpha and Beta Estimation

Factor regression is exactly the CAPM and APT estimation covered in the Portfolio Construction & Factor Models module, viewed through the regression lens introduced here. Regressing an asset's excess returns on one or more factors' excess returns produces the asset's alpha (the intercept) and beta(s) (the slope coefficients):

$$R_i - rf = \alpha + \beta_1 F_1 + \beta_2 F_2 + \dots + \beta_k F_k + \varepsilon$$

Everything from Parts 1 and 2 applies directly here. R^2 and adjusted R^2 measure how much of the asset's return variance the chosen factors explain (Section 1.3); the t-statistic on α tests whether the residual return is statistically distinguishable from zero (Section 1.4); and because factor returns are themselves time series, HAC standard errors (Section 2.4) are standard practice, since overlapping economic conditions across periods routinely produce autocorrelated residuals.

```
import statsmodels.api as sm

X = sm.add_constant(factor_returns[['Mkt-RF', 'SMB', 'HML']])
model = sm.OLS(stock_excess_returns, X).fit(cov_type='HAC', cov_kwds={'maxlags':
4})

alpha, betas = model.params['const'], model.params.drop('const')
t_alpha = model.tvalues['const'] # is the alpha statistically distinguishable
from zero?
```

Tip — A single-factor (CAPM) regression and a multi-factor regression on the same stock will generally produce different alpha estimates. If a large apparent CAPM alpha shrinks toward zero once size and value are added as controls, the 'alpha' was really just unrewarded exposure to known factors — see Common Mistake 2 below.

4.2 Pairs Trading and Hedge-Ratio Estimation

Pairs trading looks for two historically co-moving instruments — a stock and its sector ETF, or two similar companies — and takes offsetting long/short positions sized so that the combined position is insulated from the common, market-driven component of both instruments' moves. The hedge ratio that determines the offsetting position size is estimated directly by regressing one instrument's returns on the other's:

Pairs Trading: Hedge-Ratio Regression and the Resulting Spread

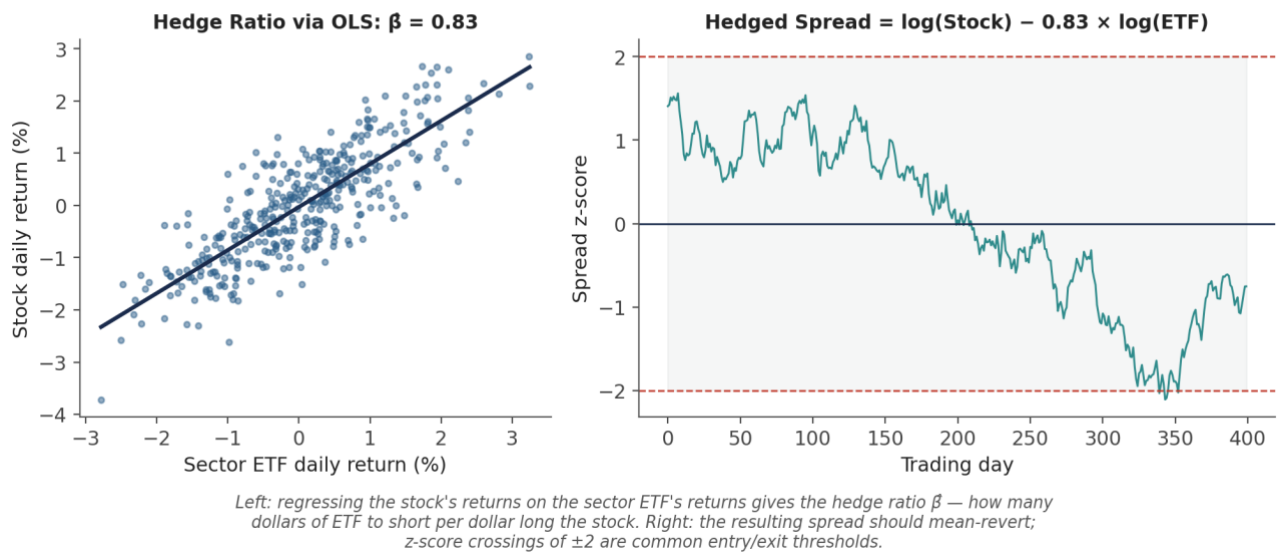


Figure 4.1 — Left: regressing a stock's daily returns on its sector ETF's daily returns gives the hedge ratio $\hat{\beta}$ — the number of dollars of ETF to short per dollar held long in the stock. Right: the resulting hedged spread, expressed as a z-score, is the signal a pairs-trading strategy actually trades — entries and exits are commonly triggered around ± 2 standard deviations.

Because the true relationship between two instruments can drift — a sector rotation, a change in one company's business mix, or a shift in market regime can all change how tightly two instruments co-move — hedge ratios are rarely estimated once on the full history and left alone. Instead, they are typically re-estimated on a rolling window, refreshing the regression as new data arrives:

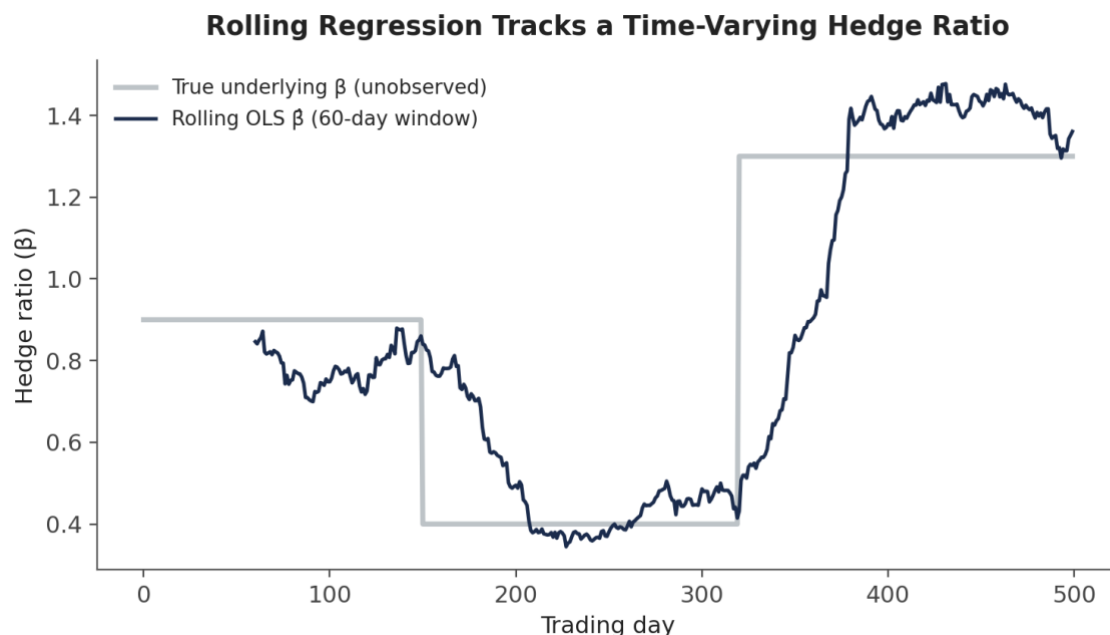


Figure 4.2 — A rolling 60-day OLS regression tracks a hedge ratio that shifts across three distinct regimes far better than a single full-sample regression could.

```
import pandas as pd
```

```
def rolling_hedge_ratio(stock_ret, etf_ret, window=60):
    cov = stock_ret.rolling(window).cov(etf_ret)
    var = etf_ret.rolling(window).var()
    return cov / var    # rolling beta, refreshed every period
```

Where diagnostics matter most here — A rolling regression is a time-series regression by construction, so its residuals — and the resulting spread the strategy actually trades — are prone to exactly the autocorrelation problem covered in Section 2.3. The practical exercise below asks you to check for it directly.

Practical Exercise

Using a single stock's daily returns over at least one year, alongside the returns of its sector ETF (or a closely related peer stock):

- Regress the stock's returns on the sector ETF's returns using OLS to estimate the hedge ratio $\hat{\beta}$, following Section 4.2.
- Record the regression's R^2 — how much of the stock's daily return variance does the sector ETF alone explain?
- Plot the residuals against fitted values (Section 2.2) and check visually for a fan or cone shape indicating heteroskedasticity.
- Plot the ACF of the residuals (Section 2.3), or compute the Durbin-Watson statistic, to check for autocorrelation.
- If either diagnostic flags a problem, re-fit the same regression using HAC standard errors (Section 2.4) and compare the resulting t-statistic and confidence interval on $\hat{\beta}$ against the default OLS output — how much does the conclusion change?
- Re-estimate the hedge ratio on a rolling 60-day window and plot it over time — does the relationship look stable, or does it drift the way Figure 4.2 illustrates?

```
import statsmodels.api as sm

X = sm.add_constant(etf_returns)
ols_model = sm.OLS(stock_returns, X).fit()
hac_model = sm.OLS(stock_returns, X).fit(cov_type='HAC', cov_kwds={'maxlags': 4})

print(ols_model.bse['etf_returns'], hac_model.bse['etf_returns']) # compare SEs
```

Common Beginner Mistakes

Common mistake — Trusting R^2 alone without checking residual diagnostics. A high R^2 says the model fits well on average; it says nothing about whether the errors are homoskedastic, uncorrelated, or normally distributed — and a regression that fails those checks can produce badly misleading standard errors even while looking impressive on R^2 alone.

Common mistake — Ignoring autocorrelated residuals when computing standard errors on time-series data. Financial return series are prone to serial correlation in regression residuals; using default OLS standard errors in this setting understates the true uncertainty and inflates apparent statistical significance. Robust or HAC standard errors are the standard fix (Section 2.4).

Common mistake — Claiming alpha without controlling for known factors. A positive intercept from a single-factor (CAPM) regression can shrink substantially, or disappear entirely, once well-known factors like size, value, or momentum are added as additional regressors — the apparent 'alpha' was partly just unrewarded exposure to a known, already-priced source of systematic risk.

Advanced Extensions

Regularised regression (Ridge and Lasso, Section 3) is the natural bridge into Level 3 machine learning, and it branches in a few directions from here. Logistic regression and other Generalised Linear Models extend the same linear-predictor machinery to classification and count outcomes rather than continuous ones. Cross-validation formalises the choice of the regularisation strength λ itself, selecting it to minimise out-of-sample error rather than guessing at a single value. And nonlinear extensions — regression trees, gradient boosting, and neural networks — relax the assumption of a linear relationship between predictors and outcome entirely, at the cost of the clean coefficient-based interpretability that has made linear regression the default first tool for nearly a century.

Professional Applications

- **Pairs trading** — hedge-ratio estimation via rolling regression is the mechanical core of nearly every statistical-arbitrage pairs strategy, determining exactly how much of the offsetting instrument to hold against the primary position.
- **Factor attribution** — performance reports across the industry are built on factor regressions, decomposing a fund's realised return into the pieces explained by market, size, value, momentum, and other systematic exposures, plus whatever is left over as alpha.
- **Hedge-ratio estimation** — beyond pairs trading, regression-based hedge ratios are used to size futures overlays, currency hedges, and any other position intended to offset a specific, measurable source of risk in a portfolio.

Where to Go Next

This module gives you the mechanical foundation — OLS, its diagnostics, and its regularised extensions — that underpins factor models, pairs trading, and the on-ramp to machine learning. Nearly everything that follows in the curriculum that involves estimating a relationship from data will use this vocabulary and these diagnostic habits without re-deriving them.

Suggested paths depending on your focus

- **Heading into research:** revisit the Portfolio Construction & Factor Models module with regression's diagnostic toolkit in hand — re-run a CAPM or Fama-French regression and check its residuals for heteroskedasticity and autocorrelation rather than taking the reported alpha at face value.
- **Heading into development:** build residual diagnostics (residual-vs-fitted plots, VIF, Durbin-Watson or an ACF check) into a shared research-pipeline utility, so every regression run through the pipeline is checked automatically rather than manually.
- **Heading into trading:** focus on Section 4.2 — implement a rolling hedge-ratio estimator and a z-scored spread, the direct mechanical core of a pairs-trading signal.